Ligue 1 Dashboard — Analysis Report

Season 2023–2024 | Power BI | Star-schema data model

This report explains the approach, data model, visuals, and storytelling of the Ligue 1 dashboard. It is based on the documentation in **Dashboard Design** and **Docs**, with **Data_Model.bim** as the source of truth for the schema. For implementation details (DAX, Power Query steps, field lists), see the referenced design documents.

Document references. Schema: `Docs/0_markdown.md`, `Data_Model.bim`. Dashboard structure: `Docs/0_POWER_BI_DASHBOARD_SPECIFICATION.md`. ETL and measures: `Docs/2_ETL_POWER_QUERY_AND_MEASURES.md`. Per-page design: `Dashboard Design/Club_View/*.md`, `Player_View/1_Player_KPI_vs_Position.md`, `Ligue-Wide_View/*.md`, `6_Visual_Charter.md`.

---

1. Overall approach

The goal was to build a single Power BI report that supports three ways of looking at the same season: by **club**, by **player**, and at **league level**. The approach is to load raw CSV/TSV and Excel files once, clean and reshape them in Power Query, load them into a star-schema model in Power BI, and then build all visuals and KPIs from that model using DAX measures. No per-club colour mapping is used; the report theme and default palette drive the look of the charts.

Data in, one model, many views. All source files (match results, player stats, transfers, stadium attendance, disciplinary sanctions, and optional club/player masters from Excel) are turned into staging tables in Power Query. Staging fixes data quality (dates, numbers, text) and keeps column names consistent. Dimensions (clubs, players, dates, matches) are built from staging and from each other; fact tables are built by merging staging to dimensions and keeping only the keys and measures needed for analysis. The result is one star schema: dimensions in the centre of the logic, facts around them, and a single place to define club names, player names, and dates. The dashboard then uses this model for Club View (five pages), Player View (one page), and Ligue-Wide View (performance, attack/defence, rivalries, sanctions).

Why this approach. A star schema fits Power BI well: filters on dimensions (e.g. one club) flow to all related fact tables, so the same measures work for “one club” or “all clubs”. Building dimensions first and facts with Merge in Power Query keeps keys aligned and avoids duplicate names or IDs. Doing the heavy work in Power Query (cleaning, keys, types) and leaving complex logic (rank, form, contribution %, rivalry KPIs) to DAX keeps the model simple and the report flexible. The documentation (Dashboard Design and Docs) describes each page and each visual so that the report can be rebuilt or extended without relying only on the .pbix file. The choice to avoid per-club colours keeps the report theme-driven and easier to maintain; club identity in charts comes from labels and slicers.

---

2. Transformations in Power Query and why

Power Query is used to load the raw files, fix known data issues, resolve keys to dimensions, and output tables that match **Data_Model.bim**. The order is: staging (clean only), then dimensions, then facts.

Staging (data quality). Each source has its own staging query. For match results, the main fixes are: normalising the match date (several formats in the CSV), setting matchday and scores to whole numbers, and handling empty attendance (null or 0). For player stats, minutes_played sometimes has " min" or spaces; goals, assists, and other numbers can have spaces or commas. Staging strips " min", trims text, and uses `Number.FromText` (and optional replace of comma by dot) so that all numeric columns load as correct types. Clean sheets and saves stay null for non-goalkeepers. For transfers, the file is tab-delimited; amount_ME has values like "22.2", "56.9M€", "67.3 M". A custom column removes "M€", " M", and spaces, then converts to number so that AmountME is decimal. Transfer dates are parsed to a single Date type. For stadium attendance, the file is semicolon-delimited and fill_rate uses a comma as decimal separator (e.g. 96,6). Staging does **not** change the column to Number before fixing; it adds a custom column that replaces comma by dot and then converts to number, so 96,6 becomes 96.6 and not 966. For sanctions, sanction_date is set to Date, suspension_matches to Whole Number (null → 0), and fine_euros to Whole Number (Integer) to match the fact table. For clubs (Excel), columns are mapped to the model names (e.g. stadium, capacity); capacity can be kept as Text if the model uses Text. For players (Excel), the two sheets (field players and goalkeepers) are appended, deduplicated on player_id, trimmed, and typed; columns include DateOfBirth, Age, Nationality, MarketValue (and optionally Salary, which is not in the model).

Why these transformations. Dates and numbers must be consistent so that merges and relationships work. Mixed date or number formats would break DimDate and measures. Fixing at staging means every downstream table (dimensions and facts) gets clean data without repeating the same logic. Using Whole Number for fine_euros and for keys avoids type mismatches with the model. Handling European decimals in fill_rate in a custom column avoids wrong values (966 instead of 96.6). Combining the two player sheets and deduplicating on player_id ensures one row per player in DimPlayer and correct enrichment for all positions.

Dimensions. DimClub is built from Staging_Clubs when the Excel file exists, or from distinct club names from all staging tables, with an index column as ClubKey. DimPlayer is built from distinct player_id, full_name, position in Staging_PlayerStats, then merged with Staging_Players to add DateOfBirth, Age, Nationality, MarketValue; PlayerKey is an index. DimDate is a generated calendar (e.g. 2023-07-01 to 2024-08-31) with DateKey; in the model, DateKey is Text for relationships. DimMatch is built from Staging_MatchResults: merge to DimClub twice to get HomeClubKey and AwayClubKey, add DateKey (from match_date, same format as DimDate), then keep MatchKey, Matchday, DateKey, HomeClubKey, AwayClubKey, Stadium, Referee, Attendance. HomeClubName and AwayClubName are not in Power Query; they are calculated columns in the model (LOOKUPVALUE from DimClub).

Facts. FactClubMatch is two rows per match (one per club), with MatchKey, ClubKey, IsHome, Points, GoalsFor, GoalsAgainst, Result, built from Staging_MatchResults and the two DimClub merges. FactPlayerSeason is one row per player per season: merge Staging_PlayerStats to DimClub and DimPlayer, add MatchesMissed (e.g. 38 – matches_played), keep all stat columns; MatchesMissed can be stored as Text in the model if needed. FactTransfer: merge to DimPlayer and DimClub (twice for departure and arrival), add TransferDateKey; output columns are TransferKey, PlayerKey, DepartureClubKey, ArrivalClubKey, TransferDateKey, AmountME, TransferType, Agent (no PlayerName in the model). FactSanction: merge to DimPlayer and DimClub, add SanctionDateKey; output is SanctionKey, PlayerKey, ClubKey, SanctionDateKey, SanctionType, Reason, SuspensionMatches, FineEuros (no PlayerName; FineEuros Integer). FactAttendance: merge to DimClub, add DateKey; optional merge to get MatchKey;

output includes ClubKey, Matchday, DateKey, MatchKey (optional), Opponent, Attendance, StadiumCapacity, FillRatePct, Weather, TemperatureC.

RivalryPair is not built in Power Query. It is a DAX calculated table in the model: one row per unordered pair of clubs (ClubKey1, ClubKey2, PairLabel), used for rivalry pages.

Relationships after load. In Power BI Model view, relationships are created from fact columns to dimension primary keys (e.g. FactClubMatch[MatchKey] to DimMatch[MatchKey], FactClubMatch[ClubKey] to DimClub[ClubKey]). For FactTransfer, both ArrivalClubKey and DepartureClubKey link to DimClub; one relationship is set inactive so that measures can use USERELATIONSHIP to choose "arrival" or "departure" context. FactAttendance can optionally link to DimMatch via MatchKey if that column is populated in the ETL. All relationships are many-to-one, single filter direction from dimension to fact.

These steps ensure that the loaded tables match Data_Model.bim in names, types, and keys, so that relationships and measures behave as designed.

---

3. Data model: diagram and justification

The model is a **star schema**: dimension tables (DimClub, DimDate, DimPlayer, DimMatch) and one calculated table (RivalryPair) are linked to fact tables (FactClubMatch, FactPlayerSeason, FactTransfer, FactSanction, FactAttendance) by foreign keys. The canonical definition is **Data_Model.bim**; the diagram below summarises it.

Diagram (relationships).

```
                    DimDate (DateKey)
                         |
      +---------------+---------------+
      |               |               |
      v               v               v
FactTransfer    FactSanction    FactAttendance
(TransferDateKey)(SanctionDateKey) (DateKey)
      |               |               |
      +-------+       +-------+       +
              |       |       |       |
              v       v       v       v
           DimClub  DimPlayer  DimMatch (DateKey -> DimDate)
              ^       ^       ^
              |       |       |
      +-------+       |       +-------+
      |               |               |
FactClubMatch   FactPlayerSeason  FactAttendance
FactTransfer    FactSanction      (MatchKey)
FactAttendance
(ClubKey)       (PlayerKey)       (MatchKey)
```

- **DimClub** (ClubKey PK): linked to FactClubMatch, FactPlayerSeason, FactAttendance, FactSanction by ClubKey; to FactTransfer by ArrivalClubKey and DepartureClubKey (two relationships, one active, one inactive for measures that need “departure” or “arrival”).

- **DimPlayer** (PlayerKey PK): linked to FactPlayerSeason, FactSanction, FactTransfer by PlayerKey.
- **DimDate** (DateKey PK, Text): linked to DimMatch by DateKey; to FactTransfer (TransferDateKey), FactSanction (SanctionDateKey), FactAttendance (DateKey).
- **DimMatch** (MatchKey PK): linked to FactClubMatch and FactAttendance by MatchKey.
- **RivalryPair** (ClubKey1, ClubKey2, PairLabel): no physical relationship; used in DAX with FILTER/CONTAINS or similar to restrict to matches between the two clubs of the selected pair.

Justification of the design.

- **Star schema:** Filters on dimensions (e.g. one club, one player, one date) propagate to all related facts. This gives a single way to “slice” the data (by club, player, time) without writing different logic for each page.

- **Two rows per match in FactClubMatch:** Each match produces one row for the home club and one for the away club. So “filter by club” immediately gives that club’s matches; Points, GoalsFor, GoalsAgainst, Result are already from that club’s perspective. This avoids repeated “if home then home_score else away_score” in measures.

- **DimMatch as a dimension:** Matches are a natural dimension (matchday, date, stadium, referee, attendance). Putting DateKey on DimMatch links the calendar to matches; facts that need “when” can use DimMatch or DimDate depending on the measure.

- **Single place for names:** Club and player names live only in DimClub and DimPlayer. Facts store only keys. This keeps naming consistent and makes it easy to fix a typo or add an attribute (e.g. Club_URL) in one place.

- **RivalryPair as a calculated table:** Rivalries are “unordered pairs of clubs”. Building this in DAX from DimMatch (distinct pairs of HomeClubKey, AwayClubKey) keeps the ETL simple and the pair list in sync with matches. Reports use PairLabel for selection and DAX measures to aggregate by pair (attendance, fill rate, wins, draws, goals, sanctions).

- **DateKey as Text:** The model uses Text DateKey for relationships (e.g. "2024-01-15"). This aligns with how dates are often stored in sources and avoids timezone/date-type issues across sources. DimDate remains the single calendar for time intelligence.

- **Fact grain:** FactClubMatch = two rows per match; FactPlayerSeason = one row per player per season; FactTransfer = one row per transfer; FactSanction = one row per sanction; FactAttendance = one row per club per matchday (home matches). Clear grains make measures (SUM, COUNT, AVERAGE) unambiguous and avoid double-counting.

---

4. Visual choices and logic per page

Club View (5 pages).
Page 1 — Season Performance: “What did we achieve?” KPIs (final rank, total points), rank evolution by matchday (line chart), win/draw/loss split, points accumulation curve, home vs away comparison, form (e.g. last 5 results). The line charts use Matchday on the X-axis so that evolution over the season is clear. Home vs away uses IsHome from FactClubMatch.
Page 2 — Squad Impact: “Who delivered?” Goals and assists leaders (bar or table), contribution % of top players, minutes distribution, age/nationality, position depth, player value vs performance scatter. These

visuals need FactPlayerSeason and DimPlayer (Position, MarketValue, etc.); contribution % is a share of team goals/assists from top N players.
Page 3 — Financial Efficiency: Squad market value vs position, transfer spend vs position, value per point, market value vs contribution, fee vs market value for arrivals, most value-efficient players. Scatter and bar charts compare money (value, spend) to sporting outcome (points, goals).
Page 4 — Transfers, availability, risk: Transfers in/out (count and/or amount), new signings' contribution, matches missed by key players (proxy for availability), cards and impact. FactTransfer and FactSanction feed these; injury data is a gap so "matches missed" is used where relevant.
Page 5 — Fans, stadium, summary: Attendance trend, capacity utilisation (fill rate), squad value rank vs final rank, executive summary. FactAttendance and DimClub are the main sources; revenue per match is N/A if no revenue data.

Player View (1 page).
Nine gauge charts: Saves, Interceptions, Shots, Shots on target, Goals, Assists, Dribbles, Tackles, Red cards. Each gauge shows the **sum** of that metric for the selected player; the **target** is the average over all players (or over same-position players if a position filter is applied). Min/max of the gauge scale follow the same scope (all players or same position). This gives a quick "above or below average" read per metric.

Ligue-Wide View.
Performance: Grouped bar (goal difference by club), stacked bars (win rate, points per match), line chart (rank at matchday for top 5 clubs). Goal difference and win rate answer "who was strong over the season?"; the line chart shows how the top 5's ranking changed over time.
Offensive–Defensive: One scatter: X = goals for per match, Y = goals against per match (inverted so "high" = better defence), size = points. Clubs fall into quadrants (e.g. dominant, fragile, pragmatic, weak). Colours from the report theme.
Rivalities (2 pages): Page 1: table of PairLabel (selector), stacked bars for average attendance and average fill rate by rivalry — "which rivalries draw the most?" Page 2: for the selected pair, HTML cards (match count, wins each side, draws, total goals), stacked bar (sanctions by rivalry with Reason as legend), and chronological match list (home/away, goals). Logic: one row per rivalry in RivalryPair; measures filter matches where both clubs are in the pair and aggregate attendance, fill rate, results, and sanctions.
Sanctions: Bar (red cards by club, Position as legend), bar (total sanctions home vs away), line (cumulative sanctions by matchday), stacked bar (sanctions by club and reason), donut (distribution by reason). The aim is to see what influences sanctions (matchday, position, home/away) and how each club behaves. Home vs away requires linking sanctions to the last match of the club on or before the sanction date (via DimMatch and DateKey/SanctionDateKey) so that each sanction is counted as "home" or "away" for the club.

Visual charter. No per-club colour mapping; charts use the report theme (e.g. theme_arena_light.json). Logos come from DimClub[Club_URL] or club_logos.json where used. Chart types (grouped bar, stacked bar, line, scatter, donut, gauge, table, HTML Content) are chosen per visual to match the question: comparison across clubs (bars), evolution over time (line), profile (scatter), proportion (donut), single-player comparison (gauges).

---

5. Storytelling: narrative thread and progression

The report is built so the user can choose the angle (club, player, or league) and then move from high-level results to detail.

Club View: Start with **Season Performance** (results and rank over time). Then **Squad Impact** (who contributed). Then **Financial Efficiency** (did spending match results?). Then **Transfers and risk** (changes and availability). Finally **Fans and summary** (stadium, value vs rank, wrap-up). So the thread is: what we achieved → who did it → how we spent → what changed and what risk we had → what it means for the club.

Player View: One page. The narrative is "how does this player compare to the average (or to his position)?" The nine gauges answer that in one place.

Ligue-Wide View: Start with **Performance** (league-wide strength: goal difference, win rate, points, top 5 evolution). Then **Offensive–Defensive** (style: attack vs defence). Then **Rivalities** (which pairs attract attention and how they compare in results and sanctions). Then **Sanctions** (factors and club behaviour). So the thread is: overall performance → playing style → rivalries → discipline.

Across the report, the same model supports "drill down": e.g. from a club's rank to its squad, then to a player's gauges, or from league sanctions to one club's sanctions. Filters (slicers, cross-highlight) tie the pages together.

---

6. Limitations and difficulties

Data gaps. Revenue per match is not in the data; Page 5 (Fans) cannot show revenue and documents it as N/A or proxy if needed. There is no dedicated injury/availability dataset; "matches missed" (e.g. 38 – matches_played) is used as a proxy. If player age, nationality, or market value are missing in the source, DimPlayer or visuals may show blanks unless an external master (e.g. players.xlsx) is provided.

Data quality. Source files had mixed date formats, European decimal separators, spaces in numbers, and text in numeric columns. These were handled in staging; if new sources are added, similar checks and transformations are needed. Club and player names must be consistent across files (trim, same spelling) so that Merge in Power Query resolves keys correctly. External clubs in transfers may not exist in DimClub; an "External" row or null is used so that FactTransfer still loads.

Model and DAX. FactTransfer has two relationships to DimClub (arrival and departure); one is inactive and used in measures with USERELATIONSHIP so that "revenue out" uses the departure side. RivalryPair has no relationship; measures use FILTER/CONTAINS on ClubKey1 and ClubKey2, which can be heavy on large datasets. Rank-at-matchday and cumulative points require either a calculated table (e.g. StandingsByMatchday) or measures that iterate over matchdays; both are documented. MatchesMissed is Text in the model in some versions; measures that use it for arithmetic may need VALUE() or a calculated column.

Visual and UX. Without per-club colours, clubs are distinguished by position in the palette or by labels; in dense charts (e.g. 20 clubs) readability can depend on labels and tooltips. The HTML Content visual for rivalry head-to-head cards is driven by a DAX measure that returns HTML; layout and zero handling (e.g. "0" instead of blank) are done in the measure.

Scope. The report covers one season (2023–2024). Multi-season comparison would require extending DimDate and the ETL (e.g. season key) and possibly adding a Season dimension. The narrative and page order are fixed; bookmarks or drill-through could be added for a more interactive path.

Practical difficulties. Aligning club and player names across CSV, Excel, and the model required consistent trimming and spelling in staging; one typo in a source can break a Merge and produce blank rows in a fact

table. European decimal separators (comma in fill_rate) had to be fixed before changing column type. The rivalry logic (unordered pairs, head-to-head counts) is implemented in DAX with FILTER over FactClubMatch and DimMatch; performance can depend on the size of the tables.

Conclusion. The Ligue 1 dashboard uses a single star-schema model (Data_Model.bim) fed by Power Query staging and dimensions/facts. Transformations in Power Query fix data quality and resolve keys so that the model is consistent and relationships work. Visuals are chosen per page to answer one main question (performance, squad, money, risk, fans, player comparison, league performance, style, rivalries, sanctions). The storytelling moves from results to drivers to context (club), or from comparison to average (player), or from league performance to style, rivalries, and discipline (league). Limitations include reliance on staging for quality and the need for USERELATIONSHIP and FILTER-based logic for transfers and rivalries.

For a PDF of approximately 10 pages, export this document with body font 11–12 pt and standard margins (e.g. 2.5 cm).